UT4
Herramientas de mapeo ORM
Hibernate

ACCESO A DATOS

2º DAM

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial 4.0 Internacional.

Contenidos

- ORM Concepto
- Hibernate
  - Arquitectura
  - Instalación
  - Proyecto Ejemplo
    - Proyecto Ejemplo XML
    - Proyecto Ejemplo JPA
  - Generación automática de código con Eclipse

Contenidos

- Sesiones y Objetos Hibernate
- Ficheros de Mapeo XML y JPA
  - Entidades
  - Identificadores
  - Propiedades
  - Asociaciones
    - One-to-One
    - Many-to-One
    - One-to-Many (Colecciones)
      - Listas Ordenadas
      - Listas Indexadas

Contenidos

- HQL – Hibernate Query Languages
  - Objeto Query
  - SELECT
  - Filtros
  - Literales
  - Operadores de Comparación
  - Operadores Lógicos
  - Operadores Aritméticos
  - Funciones de Agregación
  - Funciones sobre escalares
  - Ordenaciones
  - Agrupaciones
  - Subconsultas
  - Parámetros
  - Navegación
  - Colecciones
  - Insert,Update, Delete personalizadas

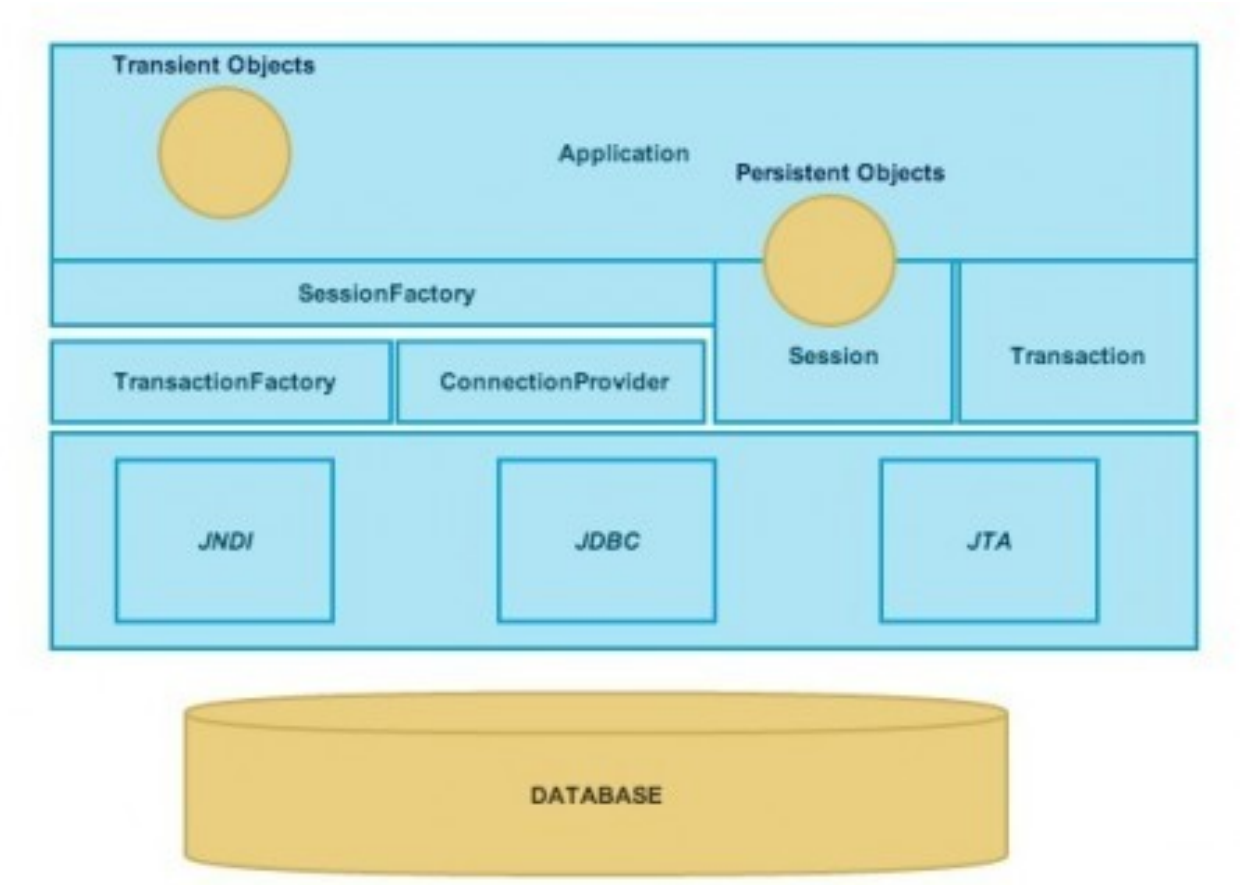
ORM Concepto

- **ORM (Object Relational Mapping)**
  - Es una técnica de programación para mapear los objetos de una aplicación a tablas relacionales y viceversa.
  - En una aplicación tradicional Java se utiliza una BD relacional para almacenar la información.
    - Se usa el API JDBC para realizar operaciones CRUD (Create, Read, Update, Delete)
    - El programador tiene que escribir código Java y consultas SQL para realizar operaciones CRUD con cada clase de la aplicación
    - Una herramienta ORM traduce directamente objetos a tablas y viceversa evitando al programador trabajar a bajo nivel con JDBC para el acceso a datos.

Hibernate

- Es un framework ORM para Java:
  - Mapeo de clases Java a tablas y viceversa
  - Gestiona transacciones y garantiza consistencia de los datos
  - Gestiona los recursos de BD de forma eficiente
  - Proporciona una capa de abstracción entre la aplicación y la BD
  - Mapeos para joins, colecciones, herencia…
  - HQL lenguaje similar a SQL pero orientado a objetos: soporta herencia, polimorfismo y asociación

Hibernate - Arquitectura



Hibernate - Arquitectura

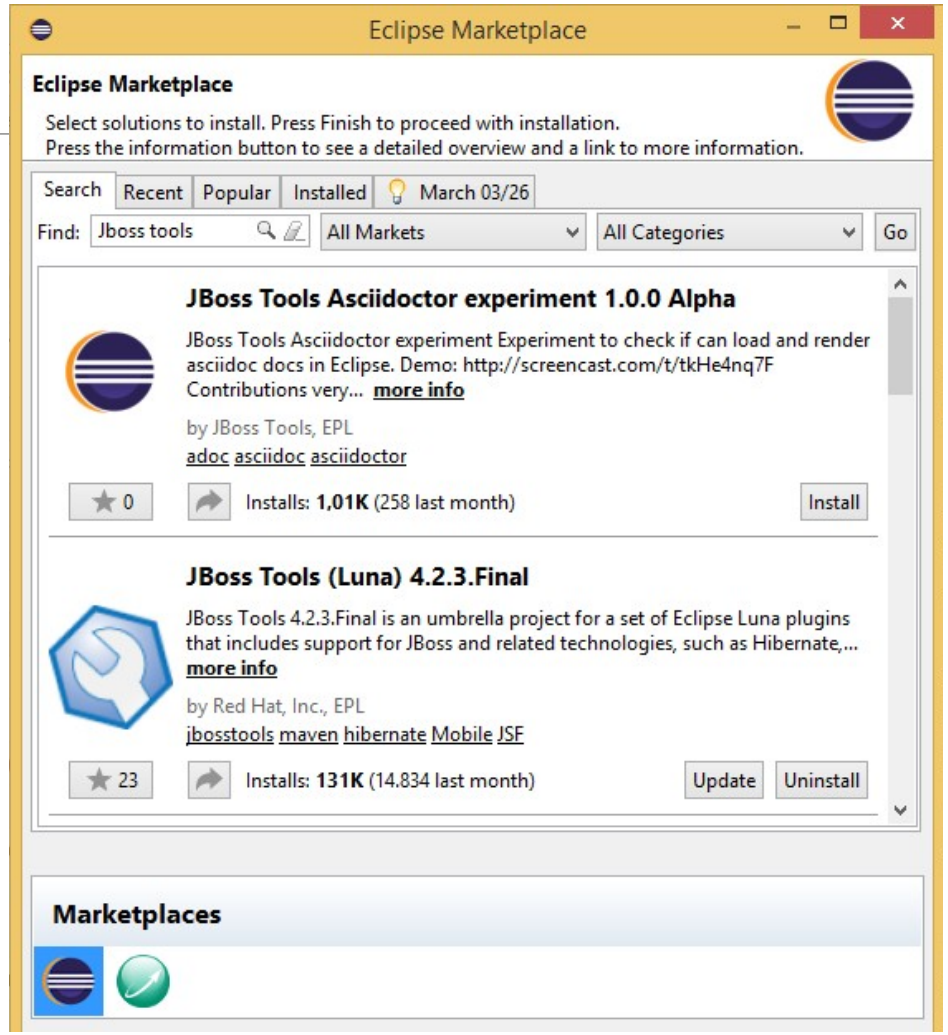
- **SessionFactory (org.hibernate.SessionFactory):** Factoría para crear sesiones para una BD
- **Session (org.hibernate.Session):** objeto envoltorio para una conexión JDBC. Es el objeto que permite a la aplicación comunicarse con la BD
- **Persistent objects**: POJOs o Java Beans que se hacen persistentes o viceversa en una sesión
- **Transient objects**: objetos instanciados en la aplicación pero no asociados a una sesión
- **Transaction (org.hibernate.Transaction)**: Gestiona transacciones aislando de JDBC y JTA
- **ConnectionProvider (org.hibernate.connection.ConnectionProvider)**: Factoria para conexiones JDBC. Gestiona DataSource y DriverManager de forma transparente
- **TransactionFactory (org.hibernate.TransactionFactory)**: Factoria para instancias de transacción

Hibernate - Instalación

- **Tenemos ya instalado las última versiones de:**
  - Xampp / devilbox
  - Eclipse (mejor versión Java EE Developers)
  - JDK
  - MySQL Connector/J
- **Vamos a instalar:**
  - Jboss Hibernate Tools (librerías Hibernate)

Hibernate - Instalación

- **Instalación Jboss Hibernate Tools**
  - Ir a Help –> Eclipse Marketplace
  - Buscar Jboss Tools
  - Click en el enlace Hibernate de Jboss Tools



Hibernate - Instalación

- **Instalación Jboss Hibernate Tools**
  - En la siguiente página seleccionamos Hibernate Tools
  - Y pulsamos Confirmar
  - Aceptamos los términos de licencia y pulsamos finalizar
  - Comprobamos que se ha instalado -> Window -> Open Perspective -> Other -> Hibernate
  - Ya tenemos Eclipse preparado

Ver proyecto Hibernate

Hibernate – Proyecto Ejemplo

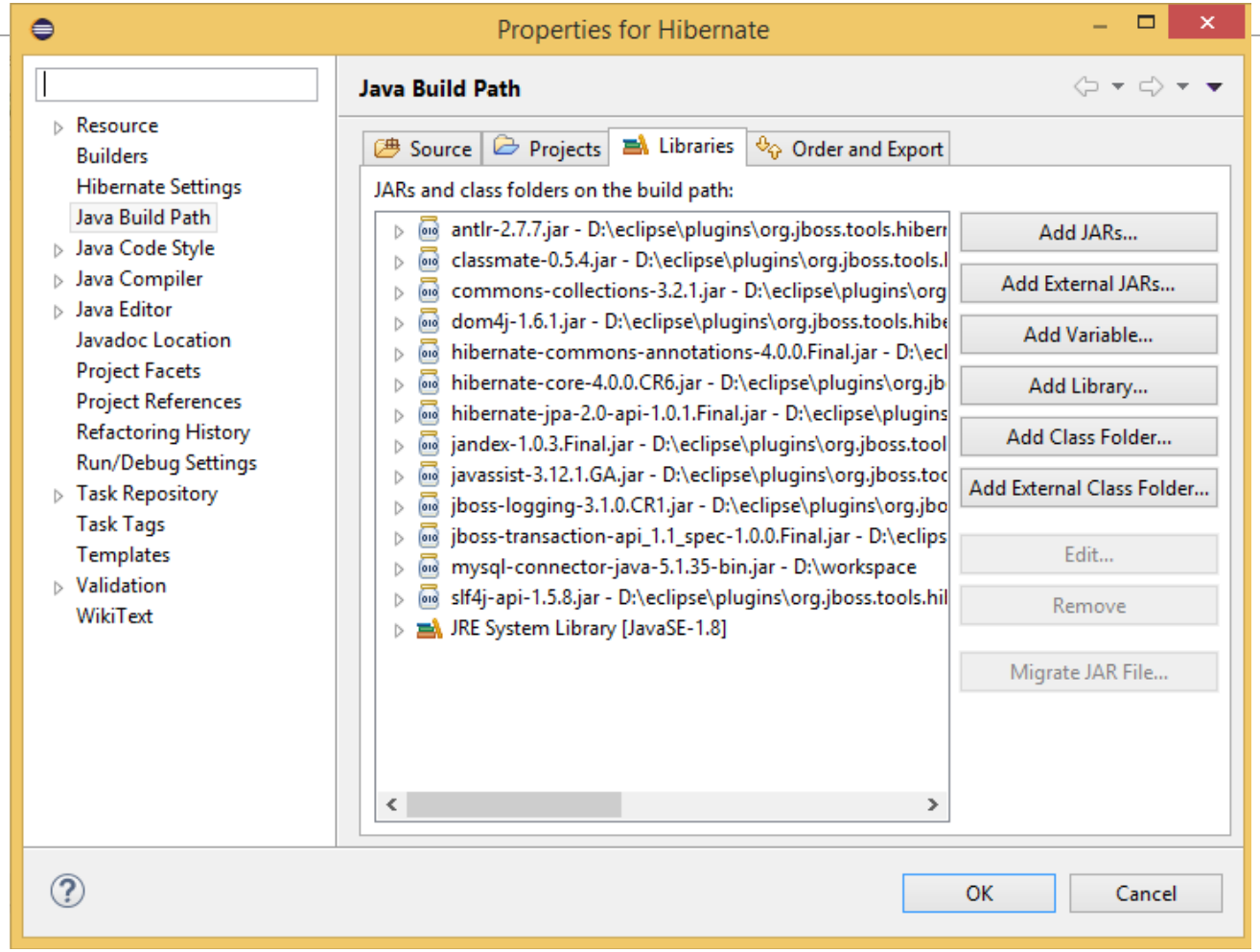
- Hibernate se basa en ficheros de configuración xml para hacer transparente la conexión a BD y para definir los mapeos objeto/tabla
- En una aplicación Hibernate vamos a tener 2 tipos configuraciones:
  - Configuración de los datos de conexión a la BD y del objeto sesión que se encarga de gestionarlos → Archivo hibernate.cfg.xml
  - Configuración para definir el mapeo entre clases (POJOs) y tablas de BD → Archivos NombreCalse.hbm.xml
    - Los mapeos también se pueden definir mediante anotaciones JPA
  - Para usar Hibernate tendremos que definir también una clase Java en la que se cree un objeto sesión con los datos de hibernate.cfg.xml y se pongan en marcha los mecanismos de mapeo

Hibernate – Proyecto Ejemplo

- Pasos a seguir:
  - Crear proyecto Java General con nombre Hibernate
    - En propiedades del proyecto → Java Build Path → Libraries → Add External Jars …
      - Añadimos todos los .jar de hibernate que previamente nos hemos descargado de la página oficial http://hibernate.org/orm/downloads/
      - Directorio\hibernate-release-5.3.13.Final\lib\required
    - Añadimos el .jar del driver mysql

Ver proyecto Hibernate

Hibernate – Proyecto Ejemplo



Ver proyecto Hibernate

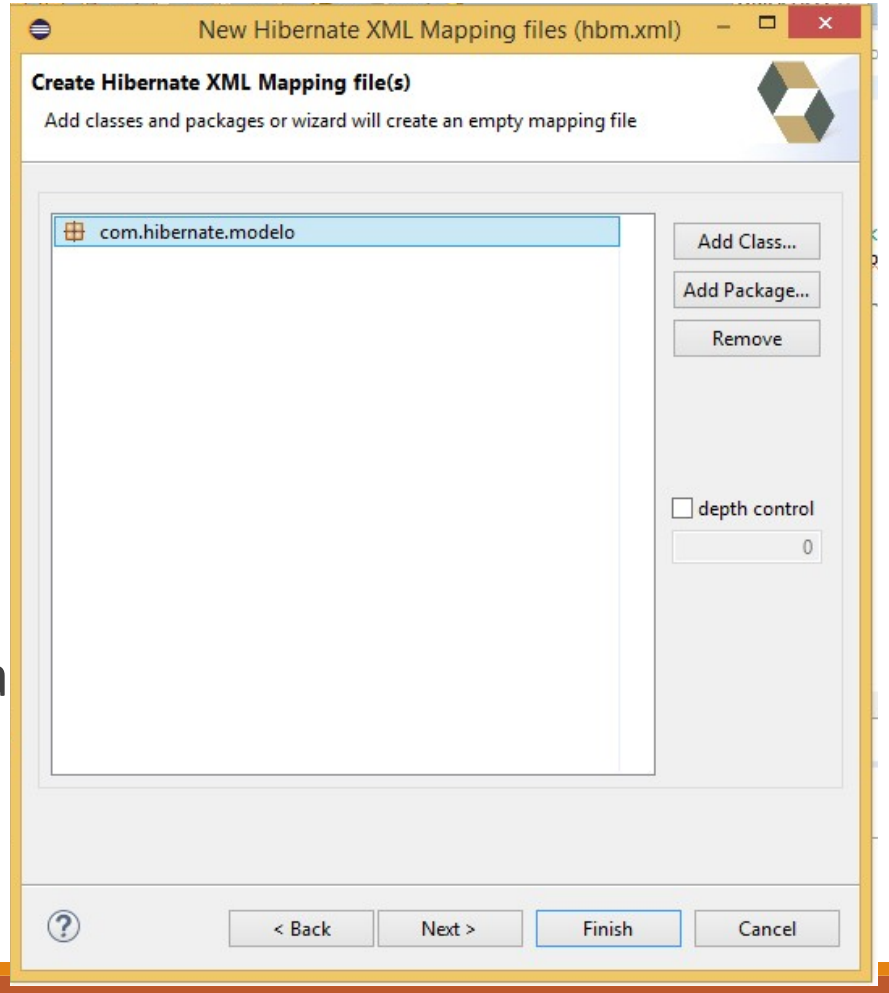
Hibernate – Proyecto Ejemplo

- Creamos los paquetes de código en src:
  - com.hibernate.main
  - com.hibernate.modelo
  - com.hibernate.utilidades
- Creamos la carpeta resources para ficheros de configuración

Ver proyecto Hibernate

Hibernate – Proyecto Ejemplo XML

- En el paquete com.hibernate.modelo creamos la clase Empleado
- Para crear el fichero de mapeo Empleado.hbm.xml:
  - File → New → Other → Hibernate XML Mapping File
  - Añadimos el paquete com.hibernate.modelo en el wizard y pulsamos finish
  - El fichero de mapeo se genera automáticamente, podemos explorarlo en modo gráfico a través de la pestaña Tree o ver el código xml en la pestaña source



Ver proyecto Hibernate

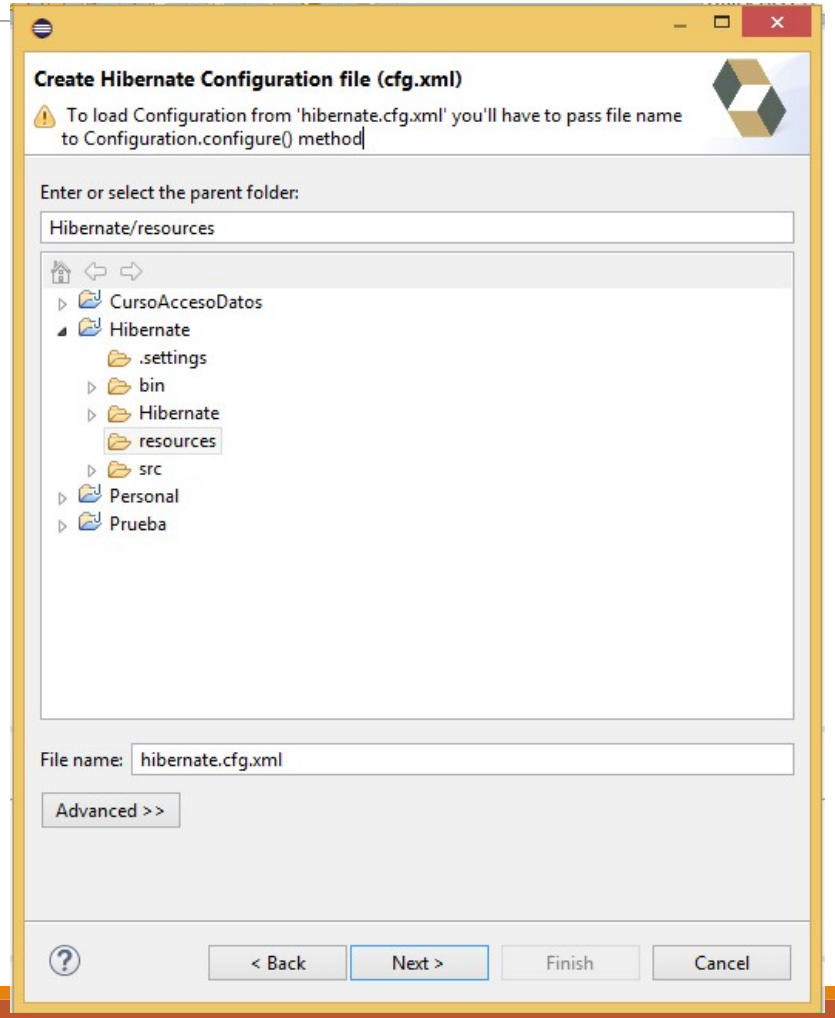
Hibernate – Proyecto Ejemplo XML

- Creamos el fichero de configuración de Hibernate

Hibernate.cfg.xml:

- File → New → Other → Hibernate Configuration File
- Se informa que para cargar el archivo de configuración desde *hibernate.cfg.xml* habrá que pasar el nombre del archivo al

método Configuration.configure(). Lo haremos luego



Ver proyecto Hibernate

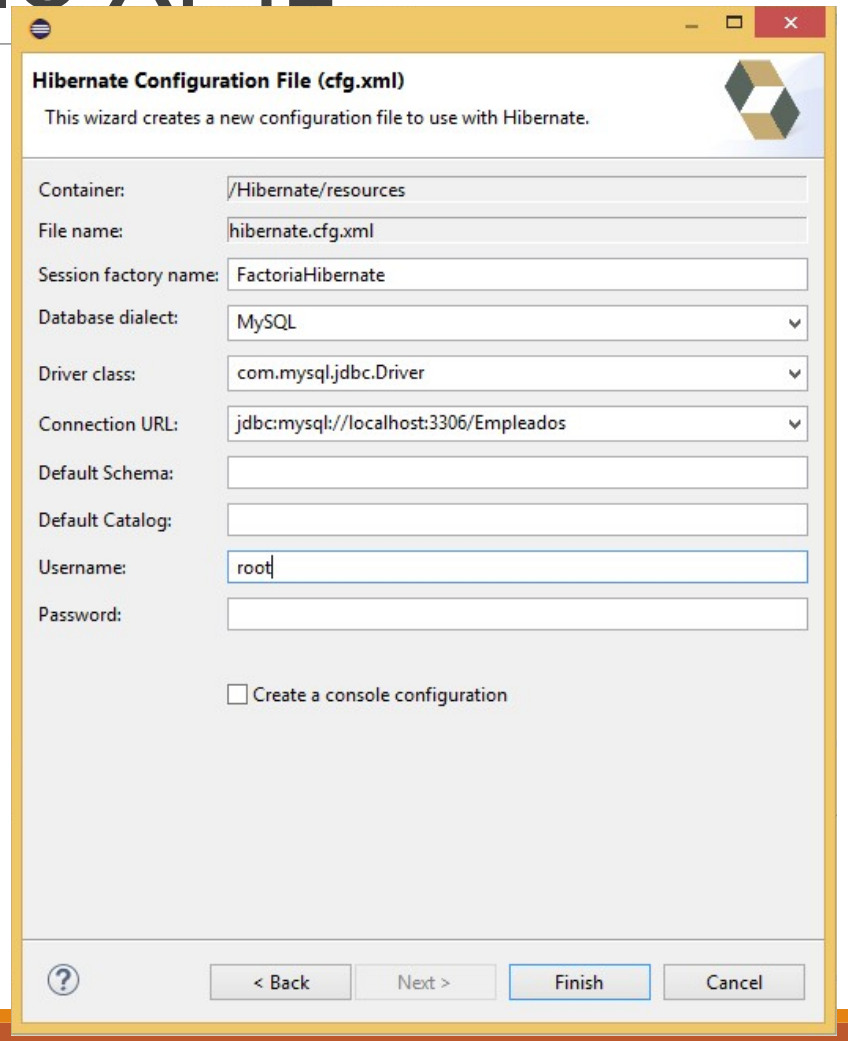
Hibernate – Proyecto Ejemplo XML

- En la siguiente pantalla rellenamos todos los datos de acceso

A BD y pulsamos finish

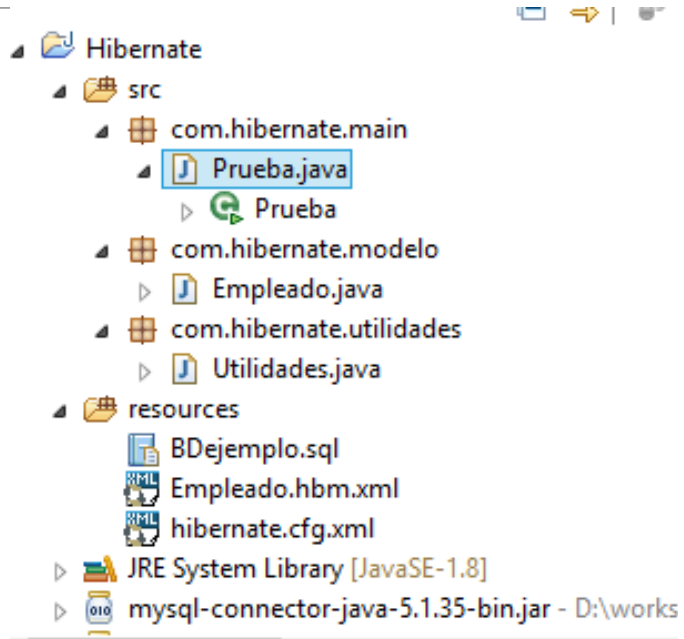
- No marcamos la casilla “Create a console configuration”
  - El fichero de configuración se genera automáticamente, podemos explorarlo en modo gráfico a través de la pestaña Tree o ver el código xml en la pestaña source
  - Al final del fichero añadimos

  <mapping resource="*Empleado.hbm.xml*" />



Ver proyecto Hibernate

Hibernate – Proyecto Ejemplo XML



- En el paquete com.hibernate.utilidades creamos la clase

Utilidades:

  - Esta clase es la responsable de crear un objeto sesión (gestiona la conexión a BD de forma transparente)

- En el paquete com.hibernate.main creamos la clase

Prueba

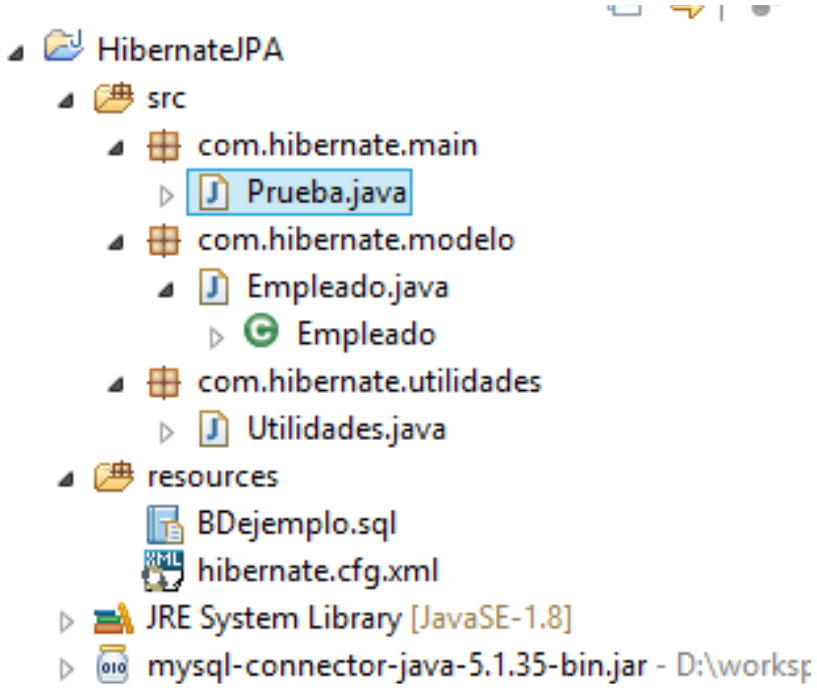
  - Esta clase obtiene un objeto sesión utilizando Utilidades.java y almacena un objeto Empleado utilizando la sesión

- Ejecutamos la clase Prueba y comprobamos como se ha almacenado el objeto empleado en BD

Ver proyecto HibernateJPA

Hibernate – Proyecto Ejemplo JPA

- En vez de ficheros de mapeo xml para las clases en Hibernate podemos utilizar anotaciones JPA



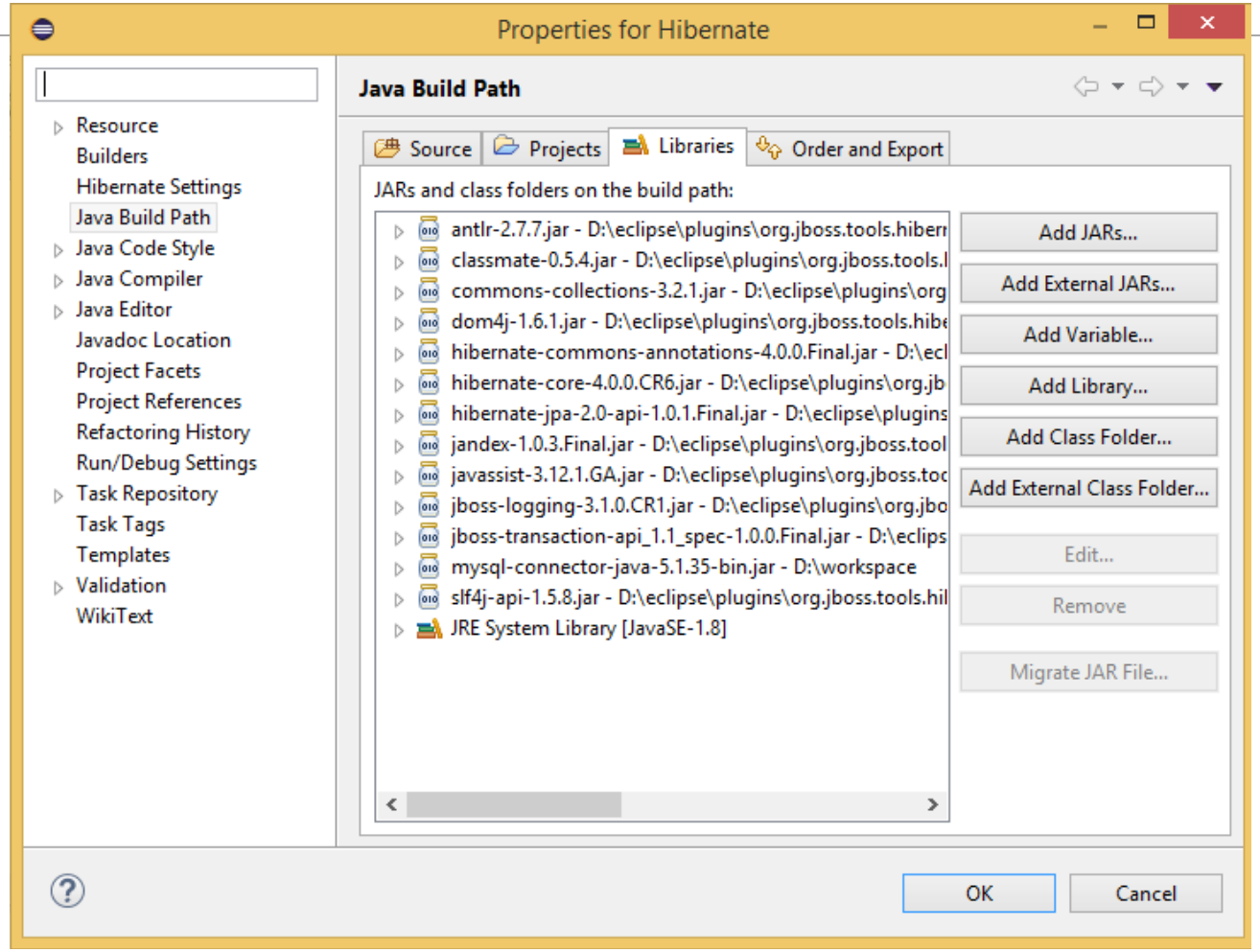
Actividad

- Dada la clase Autor.java:
  - Realiza un proyecto Hibernate con fichero de mapeo xml que almacene un objeto autor en BD
  - Realiza un proyecto Hibernate con mapeo mediante anotaciones que almacene un objeto autor en BD

Hibernate – Generación automática de código con Eclipse

- Con un IDE como Eclipse podemos automatizar la generación de código en proyectos Hibernate:
  - Fichero de configuración Hibernate
  - Ficheros de mapeo XML
  - POJOS con anotaciones JPA
- No necesitamos crear ninguna clase previamente, sólo tener creado el modelo de datos en BD
- Vamos a generar automáticamente el código para un proyecto que incluya la clase Autor.java
- La primera parte de creación del proyecto es igual. Empecemos!!

Hibernate – Generación automática de código con Eclipse



Hibernate – Generación automática de código con Eclipse

- Creamos los paquetes de código en src:
  - com.hibernate.main
  - com.hibernate.modelo
  - com.hibernate.utilidades
- Creamos la carpeta resources para ficheros de configuración

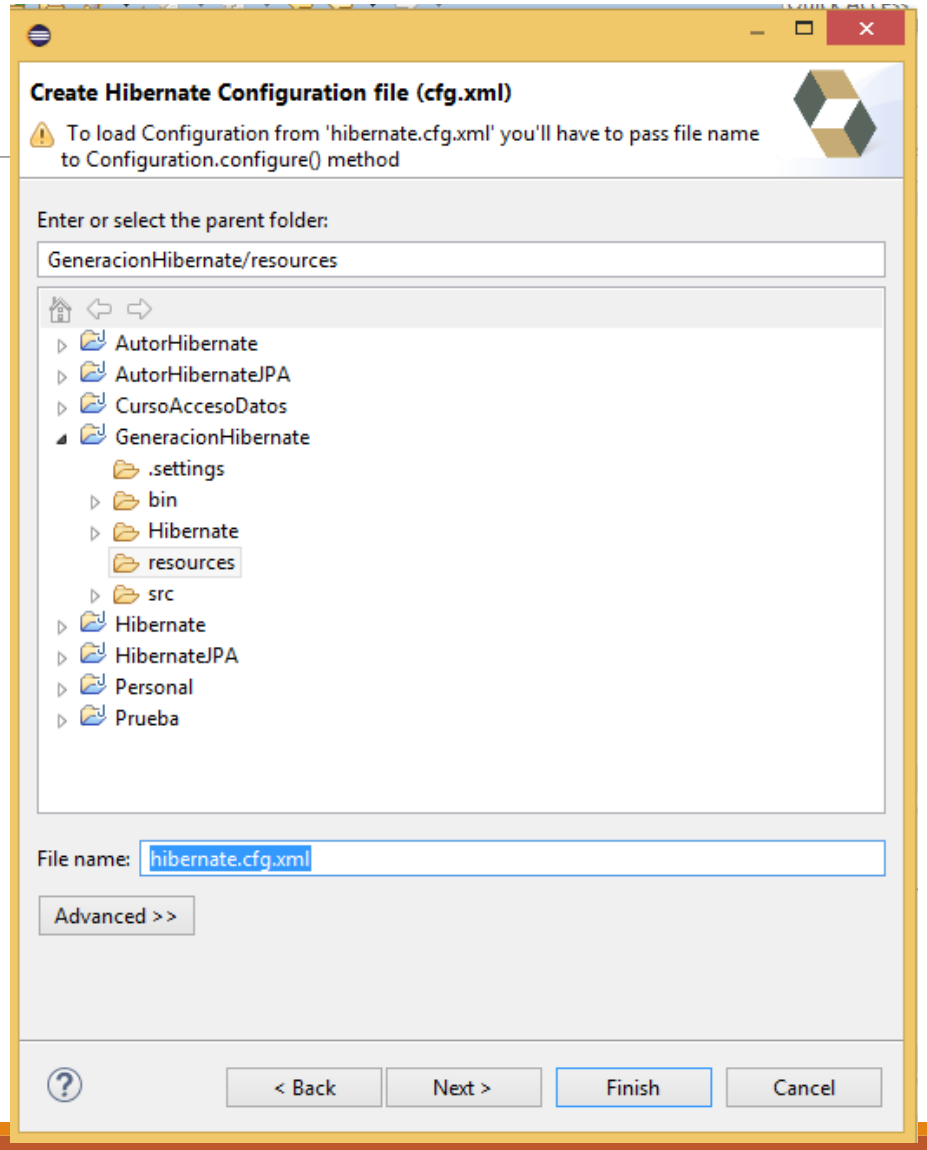
Hibernate – Generación automática de código con Eclipse

- Creamos el fichero de configuración de Hibernate

Hibernate.cfg.xml:

  - File → New → Other → Hibernate Configuration File
  - Se informa que para cargar el archivo de configuración desde *hibernate.cfg.xml* habrá que pasar el nombre del archivo al método Configuration.configure().

  Lo haremos luego

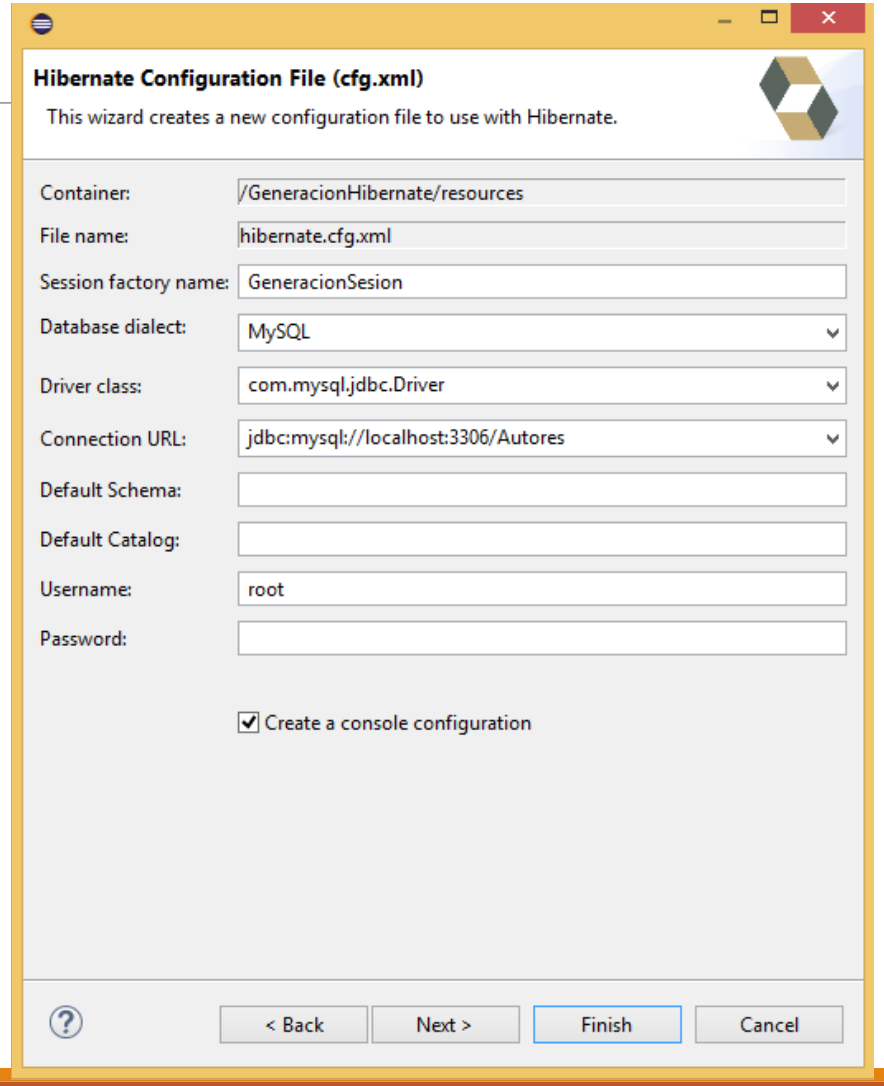


Hibernate – Generación automática de código con Eclipse

- En la siguiente pantalla rellenamos todos los datos de acceso

A BD y pulsamos finish

- **Marcamos** la casilla “Create a console configuration”
  - El fichero de configuración se genera automáticamente, podemos explorarlo en modo gráfico a través de la pestaña Tree o ver el código xml en la pestaña source



Hibernate – Generación automática de código con Eclipse

- Se lanza automáticamente el proceso de creación de la *Consola de Configuración*
- Si no se lanza accedemos a ella a través de la pestaña

Hibernate del menú de navegación de la izquierda

- **Nombre**: se dejará el mismo que del proyecto para una mejor identificación.

 - Tipo: si se usará la configuración original (*Core*),

anotaciones (jdk 1.5+) o JPA (jdk 1.5+). Se seleccionará *Core* o *Annotations*.

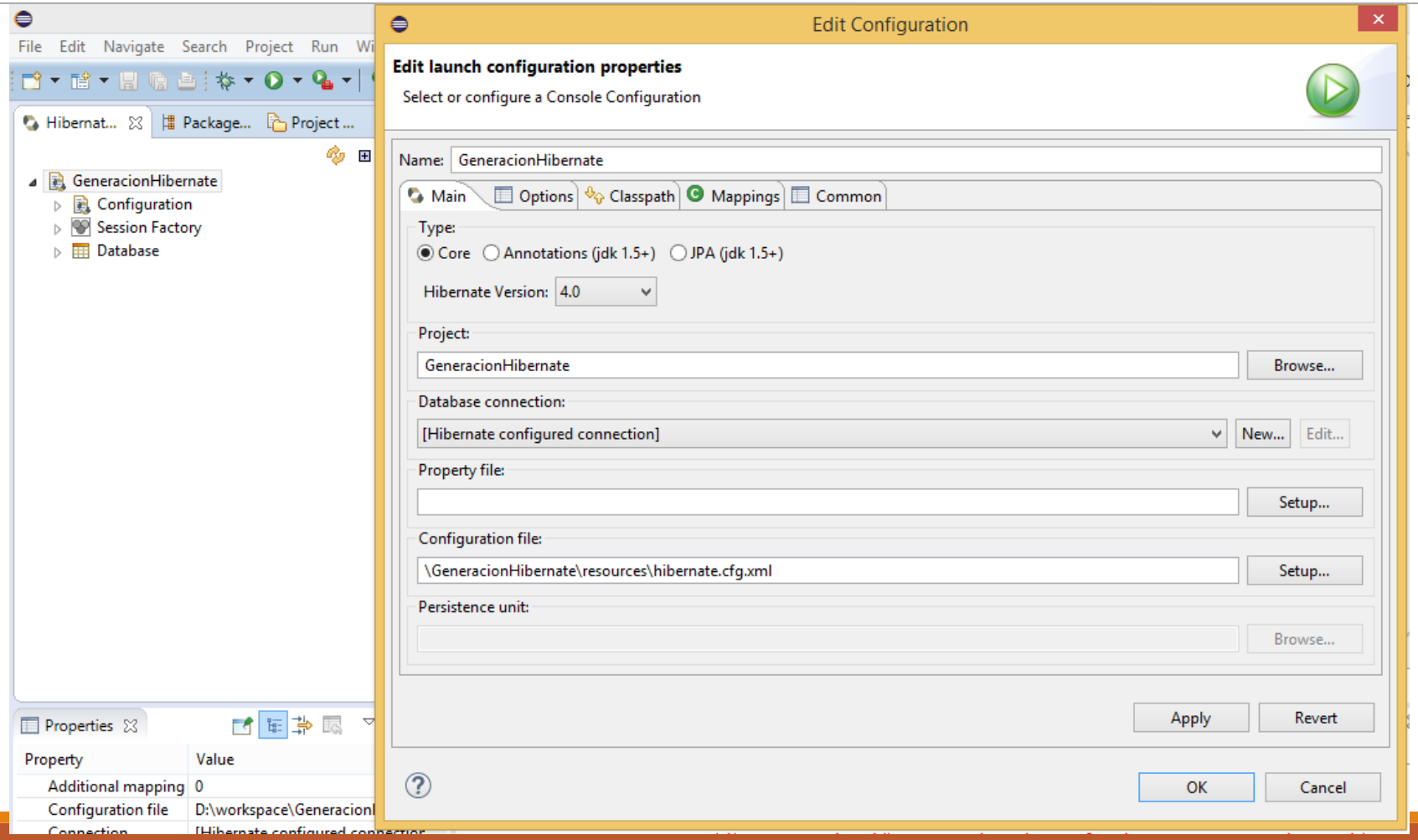
- Hibernate Version: la versión de Hibernate

- **Project**: el proyecto al que está asociada, que es el que creó esta consola

 al construir el archivo de configuración de Hibernate (hibernate.cfg.xml) en el punto anterior.

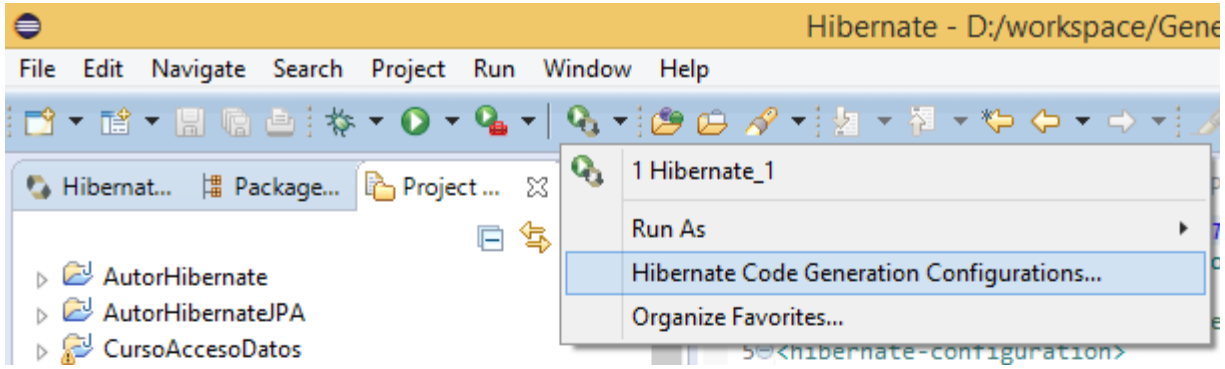
- **Database connection**: la que aparezca configurada en el archivo de configuración de Hibernate.

Hibernate – Generación automática de código con Eclipse



Hibernate – Generación automática de código con Eclipse

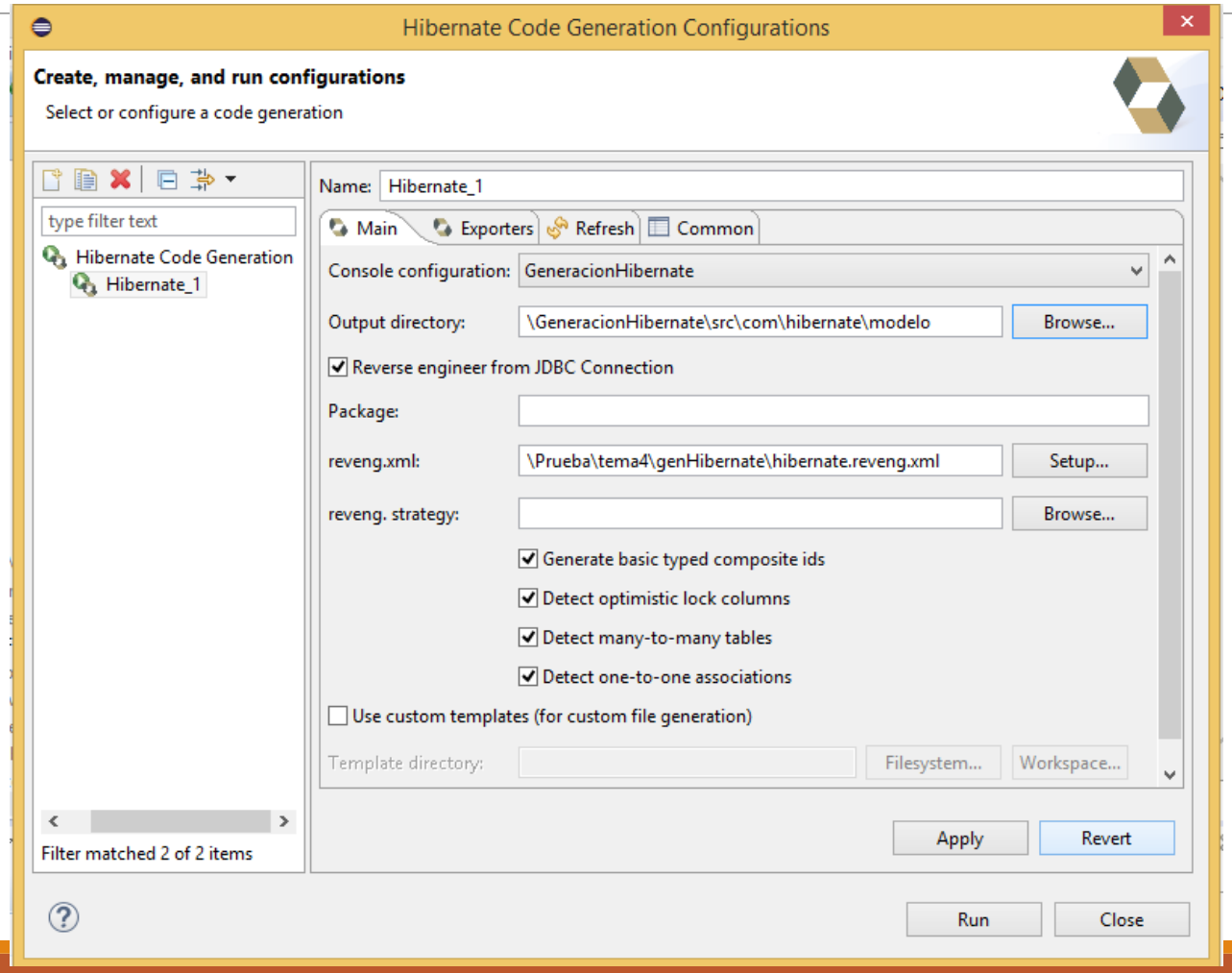
- Ahora que tenemos la consola podemos crear una “Generación de Código Asociada a ella”



Hibernate – Generación automática de código con Eclipse

- Ahora que tenemos la consola podemos crear una “Generación de Código Asociada a ella”
- En la primera pestaña se fija:
- La Consola de configuración a la que se asocia. Se debe seleccionar la creada anteriormente.
- El directorio del proyecto en el que se guardarán los archivos generados.
- El archivo de ingeniería inversa, hibernate.reveng.xml.
- Las decisiones que se adoptarán al realizar la generación:
  - Generic basic typed composite ids
  - Detect optimistic lock columns
  - Detect many-to-many tables
  - Detect one-to-one associations

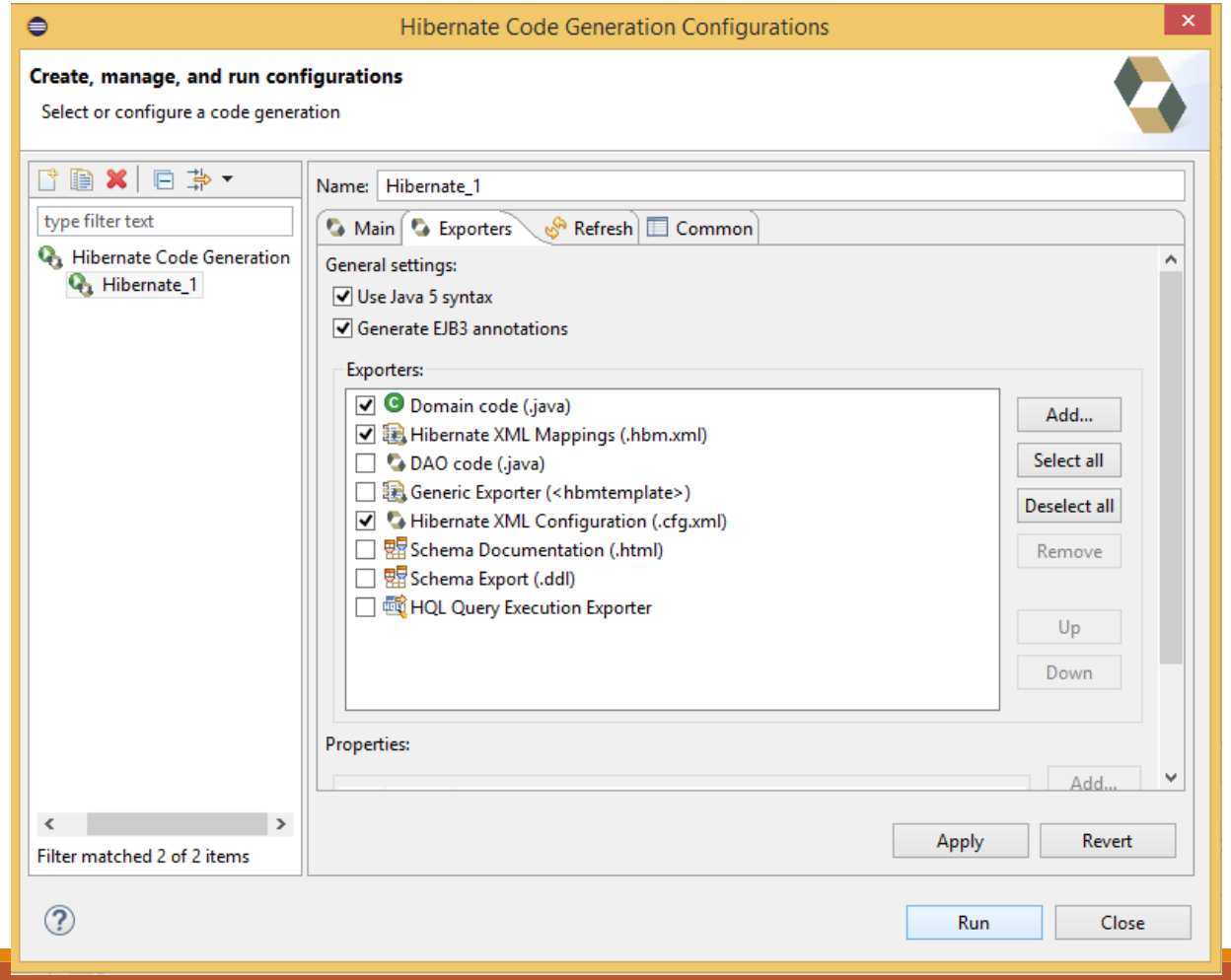
Hibernate – Generación automática de código con Eclipse



Hibernate – Generación automática de código con Eclipse

- En la pestaña Exporters, se indica qué archivos se generarán. Habría que seleccionar:
- Domain code (.java): los POJOs.
- Hibernate XML Mappings (.hbm.xml) : los archivos de mapeo.
- Hibernate XML Configuration (.cfg.xml): el archivo de configuración hibernate.cfg.xml. Se recomienda su re-generación.
- Una vez ejecutada la herramienta Consola de generación de código, se puede comprobar que, en el directorio que se eligió como salida, aparecen los archivos cuya generación se solicitó: POJOs, de mapeo, de configuración.

Hibernate – Generación automática de código con Eclipse



Hibernate – Generación automática de código con Eclipse

- El siguiente paso es construir la aplicación usando los POJOs que Hibernate ha generado por nosotros como clases de negocio:
  - En el paquete main añadimos la clase Prueba.java que ya teníamos
  - En el paquete Utilidades añadimos la clase Utilidades.java que ya teníamos encargada de crear la sesión Hibernate
  - Ejecutamos la clase Prueba y verificamos que todo funciona correctamente

Sesiones y Objetos Hibernate

- SessionFactory: es la factoría que permite crear objetos sesión. Es un objeto costoso de crear (thread-safe), pensado para compartir por todos los hilos de la aplicación. Se debe crear una sola vez al arrancar la aplicación.

- Session: es el objeto que permite comunicar con la BD. No es costoso de crear, debe usarse una sola vez (para una petición o transacción) y descartarse. Un objeto sesión no obtiene una conexión JDBC hasta que no sea necesario, hasta que no se use.

- Hibernate deshabilita el modo auto-commit, o espera que el servidor de aplicaciones lo deshabilite inmediatamente.
  - Auto-commit tiene más sentido para trabajo ad-hoc en consola SQL

Sesiones y Objetos Hibernate

- "**Ninguna comunicación con la base de datos puede darse fuera de una transacción de la base de datos**"

- Siempre usar límites de transacción claros, incluso para las operaciones de sólo lectura. Dependiendo de las capacidades de la base de datos, esto podría requerirse o no, pero no habrá ningún problema si siempre se marca explícitamente la transacción. Con seguridad, una transacción única de base de datos va a funcionar mejor que muchas transacciones pequeñas, inclusive para leer datos.
  - Nuestro código será completamente portable aunque el gestor de BD cambie

Ver Prueba.java

Sesiones y Objetos Hibernate

```
Session sess = factory.openSession();  //No consume recursos de BD hasta que no es necesario
Transaction tx = null;
try {
   tx = sess.beginTransaction();

   // realizamos operaciones
   ...
   tx.commit(); //Se hacen efectivos los cambios en BD
}
catch (RuntimeException e) {
   if (tx != null) tx.rollback();
   throw e; // o visualizamos mensaje de error
}
finally {
   sess.close();
}
```

Ver método sesionesEstados en Prueba.java

Sesiones y Objetos Hibernate

- Estados de los objetos Hibernate:
  - **Transient**: un objeto que se ha instanciado utilizando el operador new, y no está asociado a una Session de Hibernate. No tiene una representación persistente en la base de datos y no se le ha asignado un identificador.
  - **Persistent**: una instancia persistente tiene una representación en la base de datos y un identificador. Puede haber sido guardado (save) o cargado (load), sin embargo, por definición, se encuentra en el ámbito de una Session. Hibernate detectará cualquier cambio realizado a un objeto en estado persistente y sincronizará el estado con la base de datos cuando se complete la transacción
  - **Detached**: un objeto que se ha hecho persistente, pero su Session ha sido cerrada. La referencia al objeto todavía es válida, por supuesto, y la instancia desvinculada (detached) podría incluso ser modificada en este estado. Una instancia detached puede ser re-vinculada (attached) a una nueva Session más tarde, haciéndola persistente de nuevo (con todas las modificaciones). Este aspecto habilita un modelo de programación para transacciones largas que requieren tiempo-para-pensar por parte del usuario

Ver Prueba.java

Sesiones y Objetos Hibernate

- **Haciendo los objetos persistentes**:
  - Las instancias recién instanciadas son transient
  - Podemos hacer una instancia *persistente* asociándola con una sesión e invocando el método save
  - Si Cat tiene un identificador generado, el identificador es generado y asignado al cat cuando se llama a save(). Si Cat tiene un identificador assigned, o una clave compuesta, el identificador debe ser asignado a la instancia de cat antes de llamar a save()

```
DomesticCat fritz = new DomesticCat();
fritz.setColor(Color.GINGER);
fritz.setSex('M');
fritz.setName("Fritz");
Long generatedId = (Long) sess.save(fritz);
```

Ver Prueba.java

Sesiones y Objetos Hibernate

- **Haciendo los objetos persistentes**:
  - También podemos hacer un objeto persistente actualizándolo, borrándolo o incluso podemos delegar en Hibernate si la operación necesaria es guardar o actualizar

```
DomesticCat fritz = new DomesticCat();
fritz.setColor(Color.GINGER);
fritz.setSex('M');
fritz.setName("Fritz");
session.update(fritz);
```

```
DomesticCat fritz = new DomesticCat();
fritz.setColor(Color.GINGER);
fritz.setSex('M');
fritz.setName("Fritz");
session.saveOrUpdate(fritz);
```

```
session.delete(fritz);
```

Ver Prueba.java

Sesiones y Objetos Hibernate

- **Recuperando un objeto**:
  - Los métodos load() de Session le proporcionan una forma de recuperar una instancia persistente si ya conoce su identificador. load() toma un objeto clase y carga el estado dentro de una instancia recién instanciada de esa clase, en un estado persistente
  - **Cat fritz = (Cat) sess.load(Cat.class, generatedId);**
  - **load() lanzará una excepción si no hay una fila correspondiente en la base de datos**
  - Si no tienes la certeza de que existe una fila correspondiente, debe utilizar el método get(), que llama a la base de datos inmediatamente y devuelve nulo si no existe una fila correspondiente

```
Cat cat = (Cat) sess.get(Cat.class, id);
if (cat==null) {
    cat = new Cat();
    sess.save(cat, id);
}
return cat;
```

Actividad

- Dada la clase Autor.java desarrolla en eclipse con Hibernate:
  - El fichero de configuración cfg.xml de Hibernate
  - El fichero de mapeo .hbm.xml para la clase Autor
  - Una clase de prueba que proporcione 3 métodos para crear ó actualizar, recuperar, y borrar un autor.
  - Crea objetos Autor en la clase de Prueba y verifica que todo funciona correctamente

Ficheros de Mapeo Hibernate

- El mapeo entre POJO's y tablas de BD se puede definir mediante:
  - Archivos xml .hbm.xml
  - En el POJO mediante anotaciones JPA (**A lo que se tiende, más simple**)
- El mapeo es "centrado en Java": declaramos clases para hacerlas persistentes en tablas (no viceversa)
- Los principales conceptos de mapeo son:
  - Entidades
  - Identificadores
  - Propiedades
  - Componentes
  - Asociaciones

Ver Vuelo.java

Ficheros de Mapeo - Entidades

- Una entidad representa una clase Java (POJO) que vamos a hacer persistente
- Utilizamos la anotación @Entity para marcar una clase como persistente
- @Table nos permite definir en que tabla se hará persistente y restricciones de unicidad

```
@Table(name="TBL_FLIGHT",
        uniqueConstraints=
        @UniqueConstraint(columnNames={"comp_prefix", "flight_number"} ) )
```

- El mapeo equivalente en xml se realiza con el tag class
- <class name=*"com.hibernate.modelo.Empleado" table="EMPLEADO"></class>*
- El atributo name es el nombre cualificado completo de la clase

Ver Vuelo.java

Ficheros de Mapeo - Identificadores

- El identificador declara el atributo que representa la clave primaria de la tabla
- Se define con la anotación @id
  - Para indicar una clave primaria autoincremental utilizamos @GeneratedValue para indicar que es autogenerado y GenerationType.IDENTITY strategy se usa para mapear el valor autogenerado "id" y que pueda ser recuperado en el POJO por java
  - Existen otros generadores de clave según el gestor de BD que usemos: http://docs.jboss.org/hibernate/orm/4.3/manual/en-US/html/ch05.html
- En xml utilizamos el tag <id>

<id name="*id*" *type*="*int*"> (El nombre del atributo)

<column name="*ID*" /> (opcional: nombre de la columna de la clave primaria)

<generator class="*identity*" /> (clave autogenerada)

</id>

Ver Vuelo.java

Ficheros de Mapeo - Propiedades

- Las propiedades definen que atributos del POJO se harán persistentes
- @Transient indica que la propiedad no se hará persistente
- @Basic indica que la propiedad se hará persistente. Se puede omitir. Una propiedad sin anotación se hará persistente igualmente.
- Hibernate
- @Enumerated la utilizaremos para indicar que un tipo enumerado se almacene como String. Si no aparece la anotación se almacena como ordinal
  - @Enumerated(Enumtype.String)
- @Temporal para definir la precisión con la que almacenar datos temporales en BD (DATE, TIME o TIMESTAMP)
- @LOB para indicar que se haga persistente en Clob si el tipo es java.sql.Clob, Character[], char[], java.lang.String o en Blob si el tipo es java.sql.Blob, Byte[], byte[] y Serializable

Ficheros de Mapeo - Propiedades

- Podemos especificar manualmente los tipos de las propiedades con la anotación @Type, y el tipo puede ser:
  - Un tipo básico hibernate:integer, string, character, date, timestamp, float, binary, serializable, object, blob etc.
  - Un tipo básico Java: int, float, char, java.lang.String, java.util.Date, java.lang.Integer, java.sql.Clob etc
  - El nombre de una clase Java Serializable
  - El nombre de una clase definida por el usuario (tipo Personalizado)
- Si no se especifica un tipo Hibernate intenta automáticamente deducir el tipo hibernate asociado
  - Suele ser lo habitual

Actividad

- Vamos a comprobar como transforma hibernate los tipos básicos utilizando anotaciones JPA y XML.
- Lee, entiende y prueba el código del ejemplo transparencias.tiposbasicos.xml
- Lee, entiende y prueba el código del ejemplo transparencias.tiposbasicos.jpa
- Hay que fijarse especialmente en los diversas formas de almacenar un booleano, los distintos tipos de fecha y hora ,el tipo text y el tipo binary

Ficheros de Mapeo - Propiedades

- @column nos permite definir con que columnas de la tabla mapear las propiedades, podemos utilizar esta anotación sobre propiedades:
  - No anotadas
  - Anotadas con @Basic, @Temporal, @Lob
  - Los atributos más comunes en la anotación @column son:
    - name (opcional): El nombre de la columna (default nombre de la propiedad)
    - unique (opcional): establecer restricción (default false)
    - nullable (opcional): establecer como nullable (default true).
    - insertable (opcional): la columna será parte de la sentencia de inserción (default true)
    - updatable (opcional): la columna será parte de la sentencia de update(default true)
    - length (opcional): longitud(default 255)
    - precision (opcional): precision decimal (default 0)
    - scale (opcional): escala decimal (default 0)
  - @Column(name="ID", nullable=false, unique=true, length=11)

- @formula nos permite definir un calculo al obtener un valor de una propiedad. Es una propiedad de lectura.
  - @formula ("obj_length * obj_height * obj_width")

Ficheros de Mapeo - Propiedades

- En xml las propiedades se definen diferente con el tag property:

```
<property>
    name="propertyName"
    column="column_name"
    type="typename"
    update="true|false"
    insert="true|false"
    formula="SQL expression"
  unique="true|false"
    not-null="true|false"
    length="L"
    precision="P"
    scale="S"
/>
```

```
<property name="totalPrice"
  formula="( SELECT SUM (li.quantity*p.price) FROM LineItem li, Product p
            WHERE li.productId = p.productId
            AND li.customerId = customerId
            AND li.orderNumber = orderNumber )"/>
```

Actividad

- A partir de las anotaciones vistas JPA de Vuelo.java y consultando la documentación Hibernate intenta generar el archivo correspondiente Vuelo.hbm.xml
- La generación automática de código de eclipse puede serte de utilidad
- Comprueba que funciona correctamente

Actividad

- Para practicar Ejercicios1.pdf

Ficheros de Mapeo - Asociaciones

- Las relaciones de composición entre objetos (atributos de objetos que son objetos) se modelan como asociaciones en el modelo entidad-relación. Para ello se suele usar una clave foránea o una tabla para la asociación.
- Estas relaciones de asociación tenemos que definirlas explícitamente en los ficheros de mapeo de los POJOs.
- Una relación de asociación puede ser de uno a uno (@OneToOne) o de varios a uno (@ManyToOne).
- En ambas anotaciones podemos definir el parámetro cascade para propagar operaciones al objeto asociado:
  - Operaciones básicas: delete, save-update
  - Operaciones especiales: delete-orphan, all

Ficheros de Mapeo - Asociaciones

none	No se realiza ninguna acción en los objetos relacionados al hacerlo sobre el principal
save-update	Si se inserta o actualiza el objeto principal también se realizará la inserción o actualización en los objetos relacionados.
delete	Si se borra el objeto principal también se realizará el borrado en los objetos relacionados.
all	Si se realiza cualquiera de las **anteriores** acciones sobre el objeto principal también se realizará sobre los objetos relacionados.
delete-orphan	Este atributo sólo se usa si el objeto relacionado es una colección. Indica que si en la colección del objeto principal eliminamos un elemento , al persistir el objeto principal deberemos borrar de la base de datos el elemento de la colección que habíamos eliminado.
all-delete-orphan	Es la unión de los atributos all y delete-orphan [11)]

Ficheros de Mapeo - Asociaciones

- **Mapeo de una asociación (one to one ó many to one) modelada con clave foránea**
- **Anotaciones**: @OneToOne y @JoinColumn ó @ManyToOne y @JoinColumn
- @JoinColumn es opcional por defecto es el nombre de la propiedad + "_"+ el nombre de la columna marcada como @id en el objeto asociado (cargo_id en el ejemplo)
- **Tags xml**: En ambos casos many-to-one con unique a true (1-1) ó a false (1-N)
- Ver Ejemplos en transparencias.asociaciones:
  - Manytotone_xml
  - Manytoone_jpa
  - Onetoonefk_jpa
  - Onetoonefk_xml

Ficheros de Mapeo - Asociaciones

- **Mapeo de una asociación one to one compartiendo clave primaria**
- **Anotaciones:** @OneToOne, @JoinColumn, @MapsId
- **Tags xml:** one-to-one, generator class="foreign"
- Ver Ejemplos en transparencias.asociaciones:
  - onetotone_xml
  - onetoone_jpa

Ficheros de Mapeo - Asociaciones

- **Las asociaciones uno a varios se utilizan para mapear colecciones que queremos hacer persistentes.**
- Hibernate permite hacer persistentes java.util.Set, java.util.Collection, java.util.List, java.util.Map, java.util.SortedSet, java.util.SortedMap
- Utilizamos la anotación @OneToMany
- Podemos mapear las listas de dos formas:
  - Listas ordenadas en la que no almacenamos el índice en la BD
  - Listas indexadas en las que almacenamos el índice en la BD

Ficheros de Mapeo - Asociaciones

- **Listas Ordenadas**
- @OrderBy nos permite especificar porque columna ordenar los objetos de la colección antes de almacenarlos, sino se especifica por defecto es la clave primaria

```
@Entity
public class Customer {
  @Id @GeneratedValue public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }
  private Integer id;
  @OneToMany
  @OrderBy("number")
  public List<Order> getOrders() { return orders; }
  public void setOrders(List<Order> orders) { this.orders = orders; }
  private List<Order> orders;
}
```

Ficheros de Mapeo - Asociaciones

- **Listas Ordenadas**

```
-- Table schema
|-----------------| |--------------|
| Order           | | Customer |
|-----------------| |-------------|
| id              | | id           |
| number          | |--------------|
| customer_id |
|-----------------|
```

```
@Entity
public class Order {
  @Id @GeneratedValue public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }
  private Integer id;

  public String getNumber() { return number; }
  public void setNumber(String number) { this.number = number; }
  private String number;

}
```

- Ver Ejemplos en transparencias.asociaciones:
  - onetomanyord_xml
  - onetomanyord_jpa

Ficheros de Mapeo - Asociaciones

- **Listas Indexadas**
- @OrderColumn nos permite almacenar el índice de la lista en un columna especifica. El índice se almacena en la tabla que contiene la clave foránea de la asociación.

```
@Entity
public class Customer {
  @Id @GeneratedValue public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }
  private Integer id;

  @OneToMany
 @OrderColumn(name="orders_index")
  public List<Order> getOrders() { return orders; }
  public void setOrders(List<Order> orders) { this.orders = orders; }
  private List<Order> orders;
}
```

Ficheros de Mapeo - Asociaciones

- **Listas Indexadas**

```
-- Table schema
|-----------------| |--------------|
| Order           | | Customer |
|-----------------| |-------------|
| id              | | id          |
| number          | |-------------|
| customer_id |
| orders_index |
|-----------------|
```

```
@Entity
public class Order {
  @Id @GeneratedValue public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }
  private Integer id;

  public String getNumber() { return number; }
  public void setNumber(String number) { this.number = number; }
  private String number;

}
```

- Ver Ejemplos en transparencias.asociaciones:
  - onetomanyind_xml
  - onetomanyind_jpa

Actividad

- Para practicar Ejercicios2.pdf

Consultas Hibernate- HQL

- Hibernate proporciona su propio Lenguaje de Consultas HQL, es muy similar a SQL
- Es orientado a objetos, las consultas devuelven listas de objetos:
  - Soporta herencia, polimorfismo y asociaciones
  - Los tipos de datos son los de Java.
  - Las consultas son independientes del lenguaje de SQL específico de la base de datos
  - Las consultas son independientes del modelo de tablas de la base de datos.
  - Es posible tratar con las colecciones de Java.
  - Es posible navegar entre los distintos objetos en la propia consulta.

Ver **transparencias.consultas.query**

Consultas Hibernate- HQL

- Hibernate tiene el objeto **Query** que nos da acceso a todas las funcionalidades para poder leer objetos desde la base de datos.
  - **list()**: nos retorna una lista con todos los objetos que ha retornado la consulta. En caso de que no se encuentre ningún resultado se retornará una lista sin ningún elemento.
    - También se permite que las consultas retornen datos escalares en vez de clases completas
    - Un caso especial es cuando hay una única columna en el SELECT de datos escalares.
  - **uniqueResult()**:En muchas ocasiones una consulta únicamente retornará cero o un resultado. En ese caso es poco práctico que nos retorne una lista con un único elemento

Ver **transparencias.consultas.query**

Consultas Hibernate- HQL

- Las consultas a la base de datos no deberían escribirse directamente en el código sino que deberían estar en un fichero externo para que puedan modificarse fácilmente.
- Hibernate provee una funcionalidad para hacer ésto mismo de una forma sencilla. En cualquier fichero de mapeo de Hibernate se puede incluir el tag <query> con la consulta HQL que deseamos lanzar.
- Este tag tiene los siguientes datos:
  - **name**: Este atributo define el nombre de la consulta.Es el nombre que posteriormente usaremos desde el código Java para acceder a la consulta.
  - **contenido**: El contenido del tag <query> es la consulta en formato HQL que ejecutará Hibernate.
- También podemos ponerlas con JPA
  - **@NamedQuery**(name="findCustomerByName",query="from Customer c ")

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- SELECT c FROM Ciclo c ORDER BY nombre
  - Ciclo hace referencia a la clase Ciclo y NO a la tabla CicloFormativo. Nótese que la clase Java y la tabla tienen distinto nombre .
  - Es necesario definir el alias c de la clase Java Ciclo.
  - Tras la palabra SELECT se usa el alias en vez del ”*”.
  - Al ordenar los objetos se usa la propiedad nombre de la clase Ciclo en vez de la columna nombreCiclo de la tabla CicloFormativo.

- Hibernate soporta no incluir la parte del SELECT en la consulta HQL, quedando entonces la consulta de la siguiente forma:
  - FROM Ciclo

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- Las palabras clave del lenguaje **NO** son sensibles a las mayúsculas o minúsculas.
  - SELECT y select es lo mismo
- El nombre de las clases Java y sus propiedades **SI** son sensibles a las mayúsculas o minúsculas.
  - SELECT c.nombe … es correcto, SELECT c.Nombre … no lo es
- Al realizar comparaciones con los valores de las propiedades, éstas NO son sensibles a las mayúsculas o minúsculas.
- Las siguientes 2 consultas retornan los mismos objetos
  - SELECT c.nombre FROM Ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
  - SELECT c.nombre FROM Ciclo c WHERE nombre='DESARROLLO DE APLICACIONES WEB'

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **FILTROS**
- Al igual que en SQL en HQL también podemos filtrar los resultados mediante la cláusula WHERE
- Al igual que con el nombre de la clase, el nombre de los campos del WHERE siempre hace referencia a las propiedades Java y nunca a los nombres de las columnas.De esa forma seguimos independizando nuestro código Java de la estructura de la base de datos.
  - SELECT p FROM Profesor p WHERE nombre='ISABEL' AND ape1<>'ORELLANA‘

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **LITERALES**
  - **Texto**
  - El carácter para indicar un literal de texto es la comilla simple no pudiéndose usar la doble comilla.
    - SELECT p FROM Profesor p WHERE nombre='juan'
  - Si se quiere usar la comilla dentro de un literal deberemos duplicarla.
    - SELECT p FROM Profesor p WHERE ape1='perez l''andreu'
  - **Integer**
  - Para incluir un número del tipo integer simplemente se escribe dicho número.
    - SELECT tb FROM TiposBasicos tb WHERE inte=4

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **LITERALES**
  - **Long**
  - Para incluir un número del tipo long se escribe dicho número y se añade una L mayúscula al final.
    - SELECT tb FROM TiposBasicos tb WHERE long1=4L
  - **double**
  - Para representar un double se escribe el número separarando la parte decimal con un punto o se puede usar la notación científica.
    - SELECT tb FROM TiposBasicos tb WHERE double1=1.45
    - SELECT tb FROM TiposBasicos tb WHERE double1=1.7976931348623157E308

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **float**
  - Para representar un float se escribe el número separarando la parte decimal con un punto o se puede usar la notación científica pero se le añade el carácter F en mayúscula al final.
  - SELECT tb FROM TiposBasicos tb WHERE float1=1.45F
  - SELECT tb FROM TiposBasicos tb WHERE float1=3.4028235E38F
- **Fecha**
  - Para indicar una fecha la incluiremos entre comillas simples con el formato yyyy-mm-dd
  - SELECT tb FROM TiposBasicos tb WHERE dateDate='2012-07-25'
- **Hora**
  - Para indicar una hora la incluiremos entre comillas simples con el formato hh:mm:ss
  - SELECT tb FROM TiposBasicos tb WHERE dateTime='02:05:10'

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Fecha y hora**
  - Para indicar una fecha y hora la incluiremos entre comillas simples con el formato yyyy-mm-dd hh:mm:ss.millis,siendo optativos el último punto y los milisegundos .
  - SELECT tb FROM TiposBasicos tb WHERE dateTime='2012-07-25 02:05:10‘
- **Operadores de comparación**
  - Signo igual ”=”
  - Signo mayor que ”>”
  - Signo mayor que ”>=”
  - Signo mayor que ”<”
  - Signo mayor que ”<=”
  - Signo desigual ”<>”
  - Signo desigual ”!=”

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Operadores de comparación**
  - Operador ”**between**”: La expresión será verdadera si el dato de la izquierda está dentro del rango de la derecha.
    - SELECT tb FROM TiposBasicos tb WHERE inte BETWEEN 1 AND 10
  - Operador ”**in**”: La expresión será verdadera si el dato de la izquierda está dentro de la lista de valores de la derecha.
    - SELECT tb FROM TiposBasicos tb WHERE inte IN (1,3,5,7)
  - Operador ”**like**”: La expresión será verdadera si el dato de la izquierda coincide con el patrón de la derecha.Se utilizan los mismos signos que en SQL ”%” y ”_”.
    - SELECT tb FROM TiposBasicos tb WHERE stri LIKE 'H_la%'
  - Operador ”**not**”: Niega el resultado de una expresión.
  - expresión ”**is null**”: Comprueba si el dato de la izquierda es null.
    - SELECT tb FROM TiposBasicos tb WHERE dataDate IS NULL

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Operadores lógicos**
  - AND,OR, NOT
  - SELECT p FROM Profesor p WHERE nombre='ANTONIO' AND (ape1='LARA' OR ape2='RUBIO')

- **Operadores Aritméticos**
  - +,-,*,/
  - SELECT tb FROM TiposBasicos tb WHERE (((inte+1)*4)-10)/2=1

- **Funciones de agregación**
  - AVG(), SUM(),MIN(), MAX(), COUNT()
  - SELECT AVG(c.horas),SUM(c.horas),MIN(c.horas),MAX(c.horas),COUNT(*) FROM Ciclo c

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Funciones sobre escalares**:
  - UPPER(s): Transforma un texto a mayúsculas.
  - LOWER(s): Transforma un texto a minúsculas.
  - CONCAT(s1, s2): Concatena dos textos
  - TRIM(s):Elimina los espacio iniciales y finales de un texto.
  - SUBSTRING(s, offset, length): Retorna un substring de un texto.El offset empieza a contar desde 1 y no desde 0.
  - LENGTH(s): Calcula la longitud de un texto.
  - ABS(n): Calcula el valor absoluto de un número.
  - SQRT(n): Calcula la raíz cuadrada del número
  - Operador ”||” : Permite concatenar texto.
  - SELECT p.nombre || ' ' || p.ape1 || ' ' || p.ape2 FROM Profesor p WHERE Id=1001

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Ordenación**
  - Como en SQL también es posible ordenar los resultados usando ORDER BY. Su funcionamiento es como en SQL.
  - SELECT p FROM Profesor p ORDER BY nombre ASC,ape1 DESC
  - Las palabras ASC y DESC son opcionales al igual que en SQL
  - No se permite el uso de expresiones aritméticas en la cláusula ORDER BY.
- **Agrupaciones**
  - Al igual que en SQL se pueden realizar agrupaciones mediante las palabras claves GROUP BY y HAVING
  - SELECT nombre,count(nombre) FROM Profesor p GROUP BY nombre HAVING count(nombre)>1 ORDER BY count(nombre)
  - No se permite el uso de expresiones aritméticas en la cláusula GROUP BY.

Ver **transparencias.consultas.hql**

Consultas Hibernate- HQL

- **Subconsultas**

- HQL también soporta subconsultas como en SQL.
- SELECT c.nombre,c.horas FROM Ciclo c WHERE c.horas > (SELECT AVG(c2.horas) FROM Ciclo c2)

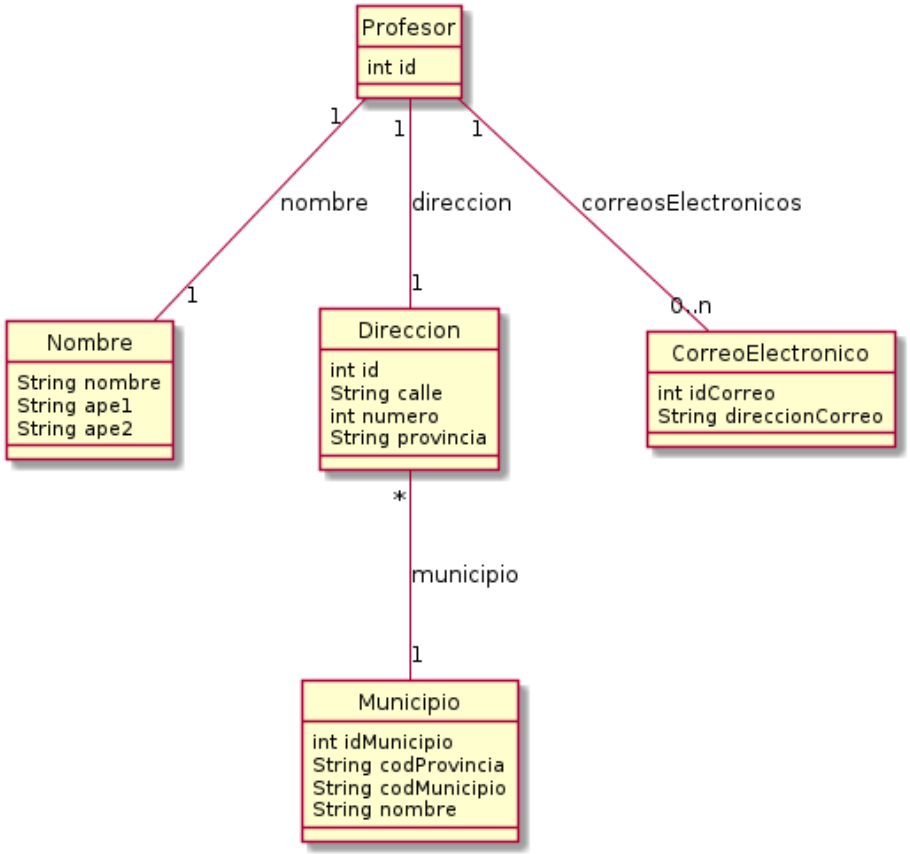
Ver **transparencias.consultas.parametros**

Consultas Hibernate- HQL

- **Parámetros**
  - En Hibernate se especifican los parámetros por nombre

  - **Nombre**: Los parámetros se definen como nombre precedidos de dos puntos ":". Esto hace que el código sea más legible y menos propenso a error. Posteriormente estableceremos a la clase Query el valor de cada uno de los parámetros con **setParameter(String name,Object val)**

Consultas Hibernate- HQL



Actividad

- Dado el diagrama de clases Java de la transparencia anterior:
  - Desarrolla un script SQL para crear las tablas en BD y rellenarlas con datos. Asegúrate de definir correctamente todas las claves foráneas para reflejar las relaciones.
  - Crea un proyecto Eclipse en el que desarrolles:
    - Los POJOs Java para las entidades de BD
    - Los ficheros Hibernate .hbm.xml para los POJOs que reflejen las relaciones de BD
    - El fichero de configuración de Hibernate .cfg.xml
    - Una clase de prueba con método main para verificar que todo funciona correctamente

Ver **transparencias.consultas.colecciones**

Consultas Hibernate- HQL

- **Navegación:** Una característica de HQL es la posibilidad de navegar por las propiedades.

  - En HQL si referenciamos una propiedad de un clase Java , esta propiedad puede no ser un valor escalar sino una referencia a otro objeto Java, que a su vez tiene más propiedades, lo que a su vez nos puede llevas hasta otro objeto Java y así sucesivamente. Es decir, podemos acceder a propiedades de Clases que no están en la cláusula FROM pero accedemos a ellas desde la clase inicial de la consulta.
  - Por ejemplo:
    - **SELECT p.nombre.ape1 FROM Profesor p**
    - Aquí vemos cómo desde objeto Profesor se navega mediante la propiedad nombre a la clase Nombre y de ahí a su propiedad ape1.
    - Los mismo se puede hacer con el nombre del municipio.
    - **SELECT p.nombre.ape1,p.direccion.municipio.nombre FROM Profesor p**

Ver **transparencias.consultas.colecciones**

Consultas Hibernate- HQL

- **Colecciones:** Otra característica de HQL es la posibilidad de tratar con colecciones dentro de la propia consulta. Veamos las funciones que tratan con colecciones.
- **SIZE(c):** Obtiene el nº de elementos de una colección.
  - SELECT p.nombre.ape1,SIZE(p.correosElectronicos) FROM Profesor p GROUP BY p.nombre.ape1
- **c IS EMPTY**: Comprueba si una colección no tiene elementos.
  - SELECT p.nombre FROM Profesor p WHERE p.correosElectronicos IS EMPTY
- **IS NOT EMPTY c**: Comprueba si una colección sí tiene elementos.
  - SELECT p.nombre FROM Profesor p WHERE p.correosElectronicos IS NOT EMPTY
- **MINELEMENT(c)**: Retorna el menor de los elementos de la colección. Para averiguar cual es el menor siempre usa el campo que hace de id.
- **MAXELEMENT(c)**: Retorna el mayor de los elementos de la colección. Para averiguar cual es el mayor siempre usa el campo que hace de id.

Ver **transparencias.consultas.iud**

Consultas Hibernate- HQL

- **Insert, Update, Delete**.

- Hibernate también permite que nosotros especifiquemos las SQL que van a usarse cuando se realiza una inserción , actualización y borrado en caso de que las que usa Hibernate tuvieran algún problema.

- El fichero de mapeo de Hibernate incluye los siguientes 3 nuevos tags para especificar las SQL

  - <sql-insert> : Contiene una sql de INSERT
  - <sql-update> : Contiene una sql de UPDATE
  - <sql-delete> : Contiene una sql de DELETE

Actividad

- Para practicar Ejercicios3.pdf

Referencias

1. Documentación Hibernate, http://docs.jboss.org/hibernate/orm/4.3/manual/en-US/html/ch01.html
2. Hibernate Tools, https://access.redhat.com/documentation/en-US/Red_Hat_JBoss_Developer_Studio/7.0/html/Hibernate_Tools_Reference_Guide/introduction.html
3. Tutorial on line, http://www.cursohibernate.es/
4. Más Ejercicios en http://cursohibernate.es/doku.php?id=ejercicios:00_start